

# *Documentation du projet Vite & Gourmand*

LAHRACH Amir

# Sommaire

|                                                           |  |
|-----------------------------------------------------------|--|
| 1. Introduction.....                                      |  |
| 1.1 Objet du document.....                                |  |
| 1.2 Portée du document.....                               |  |
| 1.3 Contexte métier.....                                  |  |
| 1.4 Vue d'ensemble de la solution.....                    |  |
| 2. Description générale.....                              |  |
| 2.1 Fonctions du produit.....                             |  |
| 2.2 Systèmes similaires.....                              |  |
| 2.3 Caractéristiques des utilisateurs.....                |  |
| 2.4 Problématique du client.....                          |  |
| 2.5 Objectifs du client.....                              |  |
| 2.6 Contraintes générales.....                            |  |
| 2.7 Exigences non fonctionnelles.....                     |  |
| 3. Réflexions technologiques initiales.....               |  |
| 3.1 Démarche.....                                         |  |
| 3.2 Back-end : PHP 8.2 / Symfony 7.4.....                 |  |
| 3.3 Front-end : Twig, Bootstrap 5, JavaScript (ES6+)..... |  |
| 3.4 Base de données relationnelle : MySQL 8.....          |  |
| 3.5 Base de données NoSQL : MongoDB 7.....                |  |
| 3.6 Outillage : Git, Docker, environnements.....          |  |
| 3.7 Hébergement : Fly.io.....                             |  |
| 4. Configuration de l'environnement de travail.....       |  |
| 4.1 Poste de développement et outils.....                 |  |
| 4.2 Environnement conteneurisé (Docker Compose).....      |  |
| 4.3 Installation du projet en local.....                  |  |
| 4.4 Environnements et variables de configuration.....     |  |

|                                                                       |  |
|-----------------------------------------------------------------------|--|
| 5. Conception.....                                                    |  |
| 5.1 Modèle conceptuel de données (MCD).....                           |  |
| 5.2 Modèle physique et scripts SQL.....                               |  |
| 5.3 Modélisation des données statistiques (MongoDB).....              |  |
| 5.4 Diagramme de cas d'utilisation.....                               |  |
| 5.5 Diagrammes de séquence.....                                       |  |
| 6. Architecture applicative.....                                      |  |
| 6.1 Architecture MVC et arborescence du projet.....                   |  |
| 6.2 Composants d'accès aux données (Doctrine ORM et ODM).....         |  |
| 6.3 Composants métier côté serveur (services, listeners).....         |  |
| 6.4 Routes API internes et échanges JSON.....                         |  |
| 7. Sécurité.....                                                      |  |
| 7.1 Authentification, rôles et contrôle d'accès.....                  |  |
| 7.2 Protection contre les vulnérabilités du web (XSS, SQL, CSRF)..... |  |
| 7.3 Données personnelles et conformité RGPD.....                      |  |
| 7.4 Veille sécurité.....                                              |  |
| 8. Tests et jeu d'essai.....                                          |  |
| 8.1 Stratégie de test.....                                            |  |
| 8.2 Jeu d'essai : calcul du prix d'une commande.....                  |  |
| 8.3 Analyse des écarts.....                                           |  |
| 9. Documentation du déploiement.....                                  |  |
| 9.1 Architecture de production : trois applications Fly.io.....       |  |
| 9.2 Préparation : Dockerfiles et configuration Fly.....               |  |
| 9.3 Étapes du déploiement (application, MySQL, MongoDB).....          |  |
| 9.4 Gestion des secrets et variables d'environnement.....             |  |
| 9.5 Initialisation des données en production.....                     |  |
| 9.6 Vérifications post-déploiement.....                               |  |

10. Annexes.....

10.1 Glossaire.....

10.2 Références (dépôt GitHub, application en production).....

# 1. Introduction

## 1.1 Objet du document

Ce document constitue la documentation technique de l'application web Vite & Gourmand. Il présente la démarche suivie tout au long du projet :

- les réflexions technologiques initiales et la justification des choix techniques
- la configuration de l'environnement de travail, la conception de la base de données et des parcours utilisateurs (MCD, diagrammes de cas d'utilisation et de séquence)
- l'architecture de l'application, les dispositions prises en matière de sécurité
- ainsi que la procédure de déploiement en production.

Il s'adresse à toute personne amenée à comprendre, maintenir ou faire évoluer l'application : développeurs de FastDev, développeurs prestataires, jury d'évaluation, ou prestataire tiers.

## 1.2 Portée du document

De plus, ce document couvre les aspects techniques du projet. Les éléments suivants font l'objet de livrables séparés et n'y sont donc pas détaillés :

- la charte graphique (palette de couleurs, typographie, wireframes et maquettes desktop/mobile) ;
- le manuel d'utilisation (présentation des parcours et identifiants de test) ;
- la documentation de gestion de projet (organisation, planification, suivi des tâches).

## 1.3 Contexte métier

Vite & Gourmand est une entreprise de traiteur bordelaise fondée il y a vingt-cinq ans, animée par deux personnes, Julie et José. Elle propose des menus pour tout type d'événement (Noël, Pâques, repas classiques, événements privés), avec une carte en constante évolution. Jusqu'à présent, les menus étaient diffusés par mail aux clients habitués.

L'entreprise souhaite gagner en visibilité et permettre à tous de consulter ses menus et de passer commande en ligne. N'ayant pas de compétences informatiques en interne, elle a confié le développement de son application web à l'entreprise FastDev, au sein de laquelle ce projet a été réalisé.

## 1.4 Vue d'ensemble de la solution

La solution livrée est une application web dynamique et responsive développée avec le framework Symfony 7.4 (PHP 8.2). Elle permet :

- aux visiteurs de découvrir l'entreprise, consulter et filtrer les menus, contacter l'entreprise et créer un compte ;
- aux utilisateurs (clients authentifiés) de commander un menu, suivre leurs commandes, les modifier ou les annuler, et déposer un avis après prestation ;
- aux employés de gérer les menus, plats et horaires, traiter les commandes (cycle de vie complet, de « acceptée » à « terminée ») et modérer les avis clients ;
- à l'administrateur de gérer les comptes employés et de consulter des statistiques de ventes (nombre de commandes et chiffre d'affaires par menu).

Les données de gestion sont stockées dans une base relationnelle MySQL 8, et les données statistiques dans une base NoSQL MongoDB, conformément aux exigences du cahier des charges. L'application est déployée en production sur la plateforme Fly.io et accessible à l'adresse : <https://vite-et-gourmand-bordeaux.fly.dev>

## 2. Description générale

### 2.1 Fonctions du produit

L'application couvre trois grands domaines fonctionnels :

- 1. Commande et suivi:** tunnel de commande avec pré-remplissage des informations du compte, calcul automatique du prix (frais de livraison hors Bordeaux : 5 € + 0,59 €/km ; remise de 10 % à partir de 5 personnes au-dessus du minimum du menu), notifications par mail (bienvenue, confirmation de commande, commande terminée, retour de matériel), suivi de l'historique des statuts, annulation et modification tant que la commande n'est pas acceptée, dépôt d'avis noté de 1 à 5.
- 2. Vitrine et catalogue:** page d'accueil présentant l'entreprise et les avis clients validés, vue globale des menus avec filtres dynamiques (prix maximum, fourchette de prix, thème, régime, nombre de personnes minimum) actualisés sans rechargement de page, vue détaillée de chaque menu (galerie d'images, plats, allergènes, conditions, stock), formulaire de contact, mentions légales et CGV.
- 3. Back-office:** espace employé (gestion des menus, plats, horaires, commandes et avis), espace administrateur (création et désactivation de comptes employés, statistiques et chiffre d'affaires par menu avec graphique comparatif alimenté par MongoDB).

### 2.2 Systèmes similaires

Le marché propose de nombreuses solutions e-commerce clés en main (Shopify, PrestaShop, WooCommerce). Elles ont été écartées car le métier du client ne correspond pas à un panier e-commerce classique : la commande porte sur une prestation datée et localisée (date, heure et lieu de livraison, nombre de convives, prêt de matériel), avec des règles de tarification spécifiques et un cycle de vie de commande propre à l'activité de traiteur. Un développement sur mesure permet de coller exactement à ces règles métier tout en gardant la maîtrise complète du code, des données et des coûts.

## 2.3 Caractéristiques des utilisateurs

| Profil                 | Compétences attendues                     | Usage                                              |
|------------------------|-------------------------------------------|----------------------------------------------------|
| Visiteur / Utilisateur | Grand public, aucune compétence technique | Consultation, commande, suivi                      |
| Employé                | Bureautique de base                       | Gestion quotidienne des menus et commandes         |
| Administrateur (José)  | Bureautique de base                       | Gestion des comptes, consultation des statistiques |

L'interface a donc été conçue pour être simple, responsive (mobile first) et accessible, conformément au RGAA.

## 2.4 Problématique du client

La diffusion des menus par mail limite la clientèle aux habitués et ne permet ni la prise de commande en ligne, ni le suivi. Chaque commande nécessite des échanges manuels (téléphone, mail), sources d'erreurs et de perte de temps pour une équipe de deux personnes.

## 2.5 Objectif du client

- Augmenter la visibilité de l'entreprise auprès de nouveaux clients
- Présenter l'ensemble des menus de façon claire, filtrable et toujours à jour
- Permettre la commande en ligne de bout en bout, avec notifications automatiques
- Piloter l'activité grâce aux statistiques de commandes et de chiffre d'affaires.

## 2.6 Contraintes générales

- Réglementaires: respect du RGPD (données personnelles des clients) et du RGAA (accessibilité) ;
- Techniques: utilisation obligatoire d'une base de données relationnelle et d'une base non relationnelle, scripts SQL de création et d'alimentation de la base livrés à la main (sans s'appuyer sur les migrations ni les fixtures) ;
- Métier: impossibilité de créer un compte administrateur depuis l'application ; un employé ne peut modifier ou annuler une commande qu'après avoir contacté le client (motif et mode de contact tracés) ;
- Livraison: l'application doit être déployée et fonctionnelle au moment de la livraison, sous peine de pénalités contractuelles.

## 2.7 Exigences non fonctionnelles

- Performance: rendu des pages côté serveur sans framework JavaScript lourd ; les filtres de menus s'actualisent sans rechargement de page avec de l'asynchrone ; les statistiques sont pré-agrégées dans MongoDB pour une lecture immédiate.

- Fiabilité: les données de production sont hébergées sur des volumes persistants Fly.io ; les scripts SQL livrés (`create.sql`, `insert.sql`) permettent de reconstruire la base à l'identique à tout moment.
- Portabilité: l'application est entièrement conteneurisée ; elle s'exécute fidèlement sur tout poste disposant de Docker, quel que soit le système d'exploitation.
- Maintenabilité et extensibilité: structure conventionnelle Symfony (contrôleurs, services, entités, dépôts), code versionné sur GitHub avec branches de fonctionnalités ; les pistes d'évolution (upselling, API mobile) sont détaillées en section 3.2.
- Accessibilité: conformité visée au RGAA (voir section 2.3).

## 3. Réflexions technologiques initiales

### 3.1 Démarche

L'énoncé n'impose aucune technologie, hormis la coexistence d'une base relationnelle et d'une base NoSQL. Les choix ont été guidés par quatre critères : l'adéquation aux besoins fonctionnels (authentification multi-rôles, mails transactionnels, règles métier riches), la robustesse et la pérennité de l'écosystème, la capacité à être déployé simplement en production, et la montée en compétence sur un socle largement utilisé en entreprise.

### 3.2 Back-end : PHP 8.2 / Symfony 7.4

Trois options ont été comparées :

- PHP natif avec PDO: suffisant pour un petit site, mais tout serait à construire à la main (sécurité, formulaires, routing, envoi de mails), avec un risque d'erreur élevé sur les points sensibles (authentification, CSRF, injections) ;
- Laravel: framework complet et populaire, tout à fait adapté, et qui repose d'ailleurs lui-même sur de nombreux composants Symfony ;
- Symfony: framework de référence, modulaire, avec des composants éprouvés pour chaque besoin du projet.

Symfony 7.4 a été retenu, pour trois familles de raisons.

Un socle mature et pérenne. Symfony bénéficie de vingt ans de développement continu (première version en 2005), d'un calendrier de versions prévisible avec des versions LTS maintenues plusieurs années, d'une promesse de rétrocompatibilité documentée et d'un processus officiel de publication des correctifs de sécurité.

Sa robustesse est attestée par le fait que des projets majeurs de l'écosystème PHP sont construits sur ses composants, à commencer par le CMS Drupal et le framework Laravel.

Pour une entreprise comme Vite & Gourmand, qui existe depuis vingt-cinq ans et investit dans un outil de long terme, cette stabilité est un critère à part entière : l'application ne repose pas sur une technologie à la mode, mais sur un standard de l'industrie.

Ne pas réinventer la roue. Chaque besoin du cahier des charges correspond à un composant éprouvé, testé par des milliers de projets en production : security-bundle (authentification, hiérarchie de rôles, contrôle d'accès), form + validator (formulaires et validation côté serveur), mailer (mails transactionnels), [symfonycasts/reset-password-bundle](#) (réinitialisation de mot de passe sécurisée), et l'ORM Doctrine pour l'accès aux données.

Le temps de développement se concentre ainsi sur ce qui a de la valeur pour le client, c'est-à-dire les règles métier du traiteur, plutôt que sur de la plomberie technique, avec un niveau de sécurité qu'un développement artisanal ne pourrait pas atteindre dans les mêmes délais. Ce choix est aussi un choix de sécurité : les protections contre les principales vulnérabilités du web sont intégrées au framework et détaillées en section 7.

Une base ouverte aux évolutions futures. L'architecture modulaire de Symfony prépare la suite de la vie du produit :

- de nouveaux modules métier peuvent être ajoutés sans refonte : un système d'upselling par exemple, où un client venant de commander recevrait automatiquement une proposition complémentaire (mise en bouche, accord mets-vins, dessert supplémentaire). Les composants Mailer et Messenger déjà en place fournissent la base d'une telle automatisation marketing ;
- si Vite & Gourmand souhaite un jour une application mobile, l'écosystème Symfony (notamment API Platform) permet d'exposer une API à partir de la même base de code et des mêmes entités ;
- la structure conventionnelle du framework (arborescence, injection de dépendances, séparation contrôleurs / services / entités) garantit que tout développeur connaissant Symfony peut reprendre le projet sans phase de rétro-ingénierie, un point décisif pour FastDev comme pour son client, qui n'est pas lié à un développeur en particulier ;
- enfin, Symfony étant l'un des frameworks les plus demandés sur le marché de l'emploi français, recruter ou sous-traiter la maintenance ne posera pas de difficulté.

### **3.3 Front-end : Twig, Bootstrap 5, JavaScript (ES6+)**

Une architecture SPA (React, Vue) a été envisagée puis écartée : elle aurait imposé de développer et sécuriser une API complète, pour un bénéfice limité sur un site majoritairement constitué de pages de contenu. Le rendu côté serveur avec Twig a été préféré pour trois raisons :

- SEO : le HTML est intégralement rendu par le serveur, donc immédiatement indexable par les moteurs de recherche, un point décisif pour l'objectif de visibilité de Julie et José ;
- simplicité et accessibilité : des pages HTML classiques facilitent la conformité RGAA ;
- performance : pas de framework JavaScript lourd à charger côté client.

Le JavaScript est réservé aux interactions dynamiques exigées par le cahier des charges, notamment le filtrage des menus sans rechargement de page, réalisé en ES6 via des appels fetch vers des routes API internes renvoyant du JSON. Bootstrap 5 assure la grille responsive et

les composants d'interface, personnalisés selon la charte graphique. Les assets sont compilés avec Webpack Encore, l'outil d'intégration officiel de l'écosystème Symfony.

### **3.4 Base de données relationnelle : MySQL 8**

Les données de gestion (utilisateurs, menus, plats, commandes, avis...) sont fortement structurées et liées entre elles : un modèle relationnel s'impose. Parmi les options proposées (MySQL, MariaDB, PostgreSQL), MySQL 8 a été retenu : SGBD le plus répandu de l'écosystème PHP, parfaitement supporté par Doctrine, disposant d'une image Docker officielle et facilement hébergeable sur Fly.io.

L'encodage utf8mb4 est utilisé pour un support complet des caractères accentués. L'accès aux données passe par Doctrine ORM (requêtes préparées systématiques, protection native contre les injections SQL).

### **3.5 Base de données NoSQL : MongoDB 7**

Le cahier des charges impose que les statistiques de l'espace administrateur (commandes par menu, chiffre d'affaires) proviennent d'une base non relationnelle. MongoDB a été choisi : c'est la base documentaire la plus mature, intégrée à Symfony via le bundle officiel Doctrine MongoDB ODM, ce qui permet de manipuler les documents avec la même logique que les entités Doctrine.

Les données statistiques sont dénormalisées dans une collection dédiée, alimentée automatiquement depuis l'application, un cas d'usage typique du NoSQL : des lectures agrégées rapides sans jointures.

### **3.6 Outillage : Git, Docker, environnements**

- Git / GitHub : gestion de versions avec une branche main (production), une branche develop (intégration) et une branche par fonctionnalité, fusionnée dans develop après test, conformément aux bonnes pratiques exigées par l'énoncé ; Docker &
- Docker Compose : l'environnement de développement complet (serveur web, PHP, MySQL, MongoDB, serveur mail de test) est conteneurisé, garantissant un environnement identique et reproductible sur n'importe quel poste, et proche de la production ;
- MailHog : serveur SMTP de test capturant tous les mails envoyés en développement, évitant tout envoi réel accidentel

### **3.7 Hébergement : Fly.io**

Parmi les plateformes suggérées (Fly.io, Heroku, Azure, Vercel) : Vercel est orienté front-end JavaScript et ne convient pas à une application PHP avec bases de données ; Heroku n'offre plus d'offre gratuite viable, Azure est surdimensionné pour ce projet. Fly.io a été retenu :

- déploiement à partir de nos propres Dockerfiles (cohérence totale avec l'environnement de développement)
- possibilité d'héberger l'application et ses deux bases de données sous forme de trois applications distinctes communiquant par réseau privé interne, gestion des secrets intégrée, et HTTPS automatique.

## 4. Configuration de l'environnement de travail

### 4.1 Poste de développement et outils

Le développement a été réalisé sur un poste unique sous Windows 11, avec les outils suivants :

- Visual Studio Code comme éditeur principal, complété d'extensions adaptées au projet (support PHP avec Intelephense, coloration des templates Twig, intégration Docker et Git) ;
- Git pour la gestion de versions, avec GitHub comme dépôt distant ;
- Docker Desktop (moteur WSL2) pour l'exécution de l'environnement conteneurisé décrit en 4.2 ;
- un navigateur équipé de ses outils de développement pour l'inspection du rendu, du responsive et des requêtes réseau. Certains outils sont également requis directement sur le poste, car la compilation des assets et les commandes Symfony s'exécutent hors des conteneurs : PHP 8.2 ou supérieur, Composer 2, Node.js 18 ou supérieur et npm 9 ou supérieur.

Certains outils sont également requis directement sur le poste, car la compilation des assets et les commandes Symfony s'exécutent hors des conteneurs : PHP 8.2 ou supérieur, Composer 2, Node.js 18 ou supérieur et npm 9 ou supérieur.

### 4.2 Environnement conteneurisé (Docker Compose)

Plutôt que d'installer les services (serveur web, bases de données, serveur mail) directement sur le poste, l'environnement de développement complet est décrit dans un fichier docker-compose.yml et s'exécute dans cinq conteneurs isolés :

| Service | Image / construction | Rôle                                                                                                       | Accès local                                               |
|---------|----------------------|------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| nginx   | nginx 1.22-alpine    | Serveur web : sert les fichiers statiques du dossier public/ et transmet les requêtes PHP au conteneur php | <a href="http://localhost:8080">http://localhost:8080</a> |
| php     | Image personnalisée  | Exécution de l'application interne                                                                         |                                                           |

| Service | Image / construction          | Rôle                                                                    | Accès local                                                                 |
|---------|-------------------------------|-------------------------------------------------------------------------|-----------------------------------------------------------------------------|
|         | (php:8.3-fpm-alpine)          | Symfony via PHP-FPM                                                     |                                                                             |
| mysql   | Image personnalisée (MySQL 8) | Base de données relationnelle, encodage utf8mb4                         | port 3307                                                                   |
| mongodb | mongodb-atlas-local           | Base de données NoSQL (statistiques)                                    | port 27017                                                                  |
| mailhog | mailhog/mailhog               | Serveur SMTP de test : intercepte tous les mails émis par l'application | interface web sur <a href="http://localhost:8025">http://localhost:8025</a> |

L'image PHP est construite à partir d'un Dockerfile propre au projet, qui installe les extensions nécessaires : pdo\_mysql et mysqli pour MySQL, l'extension mongodb compilée via PECL pour la base NoSQL, ainsi que Composer 2.

Ce choix apporte trois garanties. La reproductibilité : tout poste équipé de Docker obtient exactement le même environnement, quelle que soit sa configuration. L'isolation : aucun service n'est installé en dur sur la machine, et les données persistent dans des volumes Docker nommés, indépendants du cycle de vie des conteneurs.

La proximité avec la production : l'application est déployée sur Fly.io à partir de Dockerfiles similaires, ce qui réduit fortement le risque de comportement divergent entre le développement et la production (voir section 9). Le code source est monté dans le conteneur PHP : toute modification d'un fichier est prise en compte immédiatement, sans reconstruction d'image.

### 4.3 Installation du projet en local

La procédure complète, également décrite dans le README.md du dépôt, est la suivante :

#### 1. Cloner le dépôt

```
git clone https://github.com/Hatenaa/Vite-Gourmand.git
cd Vite-Gourmand
```

#### 2. Installer les dépendances PHP et JavaScript

```
composer install
npm install
npm run build
```

La commande `npm run build` compile les assets via Webpack Encore : feuilles de style Sass s'appuyant sur Bootstrap 5, icônes Bootstrap Icons, ainsi que les bibliothèques JavaScript du projet (Chart.js pour le graphique des statistiques, noUiSlider pour le filtre de fourchette de prix).

### 3. Configurer l'environnement

```
cp .env .env.local
```

Le fichier `.env.local` est ensuite édité pour renseigner les valeurs propres au poste (identifiants MySQL et MongoDB, configuration du mailer), comme détaillé en 4.4.

### 4. Démarrer les conteneurs

```
docker compose up -d --build
```

### 5. Initialiser la base de données

Deux possibilités : exécuter les migrations Doctrine (flux de développement) ou rejouer les scripts SQL livrés avec le projet (`sql/create.sql` puis `sql/insert.sql`), qui reconstruisent le schéma et les données de démonstration à l'identique.

```
php bin/console doctrine:migrations:migrate
```

### 6. Vérifier l'installation

L'application est accessible sur <http://localhost:8080> et l'interface MailHog sur <http://localhost:8025>. Les identifiants des comptes de démonstration (administrateur, employé, client) sont fournis dans le manuel d'utilisation.

## 4.4 Environnements et variables de configuration

La configuration suit le mécanisme standard de Symfony, qui sépare strictement le code de la configuration d'infrastructure :

- `.env` versionné) : valeurs par défaut et liste exhaustive des variables attendues, sans aucun secret réel ;
- `.env.local` (non versionné, exclu par `.gitignore`) : valeurs réelles propres au poste de développement ;
- variables d'environnement système : prioritaires sur les fichiers, c'est par ce canal que la production est configurée (secrets Fly.io, voir section 9.4).

Les principales variables du projet :

| Variable                                           | Rôle                                                                                         |
|----------------------------------------------------|----------------------------------------------------------------------------------------------|
| <code>APP_ENV</code> / <code>APP_SECRET</code>     | Mode d'exécution ( <code>dev</code> ou <code>prod</code> ) et clé secrète du framework       |
| <code>DATABASE_URL</code>                          | Chaîne de connexion MySQL (hôte, identifiants, base, version serveur, <code>utf8mb4</code> ) |
| <code>MONGODB_URI</code> / <code>MONGODB_DB</code> | Connexion à la base MongoDB et nom de la base de statistiques                                |

| Variable                               | Rôle                                                                                                            |
|----------------------------------------|-----------------------------------------------------------------------------------------------------------------|
| <code>MAILER_DSN</code>                | Transport d'envoi des mails ; pointe vers MailHog en développement, vers le fournisseur SMTP réel en production |
| <code>MAILER_FROM/CONTACT_EMAIL</code> | Adresse d'expédition des mails automatiques et adresse de réception des demandes de contact                     |
| <code>MYSQL_*</code>                   | Identifiants injectés dans le conteneur MySQL par Docker Compose                                                |

Ce découpage garantit qu'aucun secret n'est présent dans le dépôt Git public : les fichiers versionnés ne contiennent que des valeurs factices (`change_me`), et chaque environnement (poste local, production) porte ses propres valeurs réelles.

## 5. Conception

### 5.1 Modèle conceptuel de données (MCD)

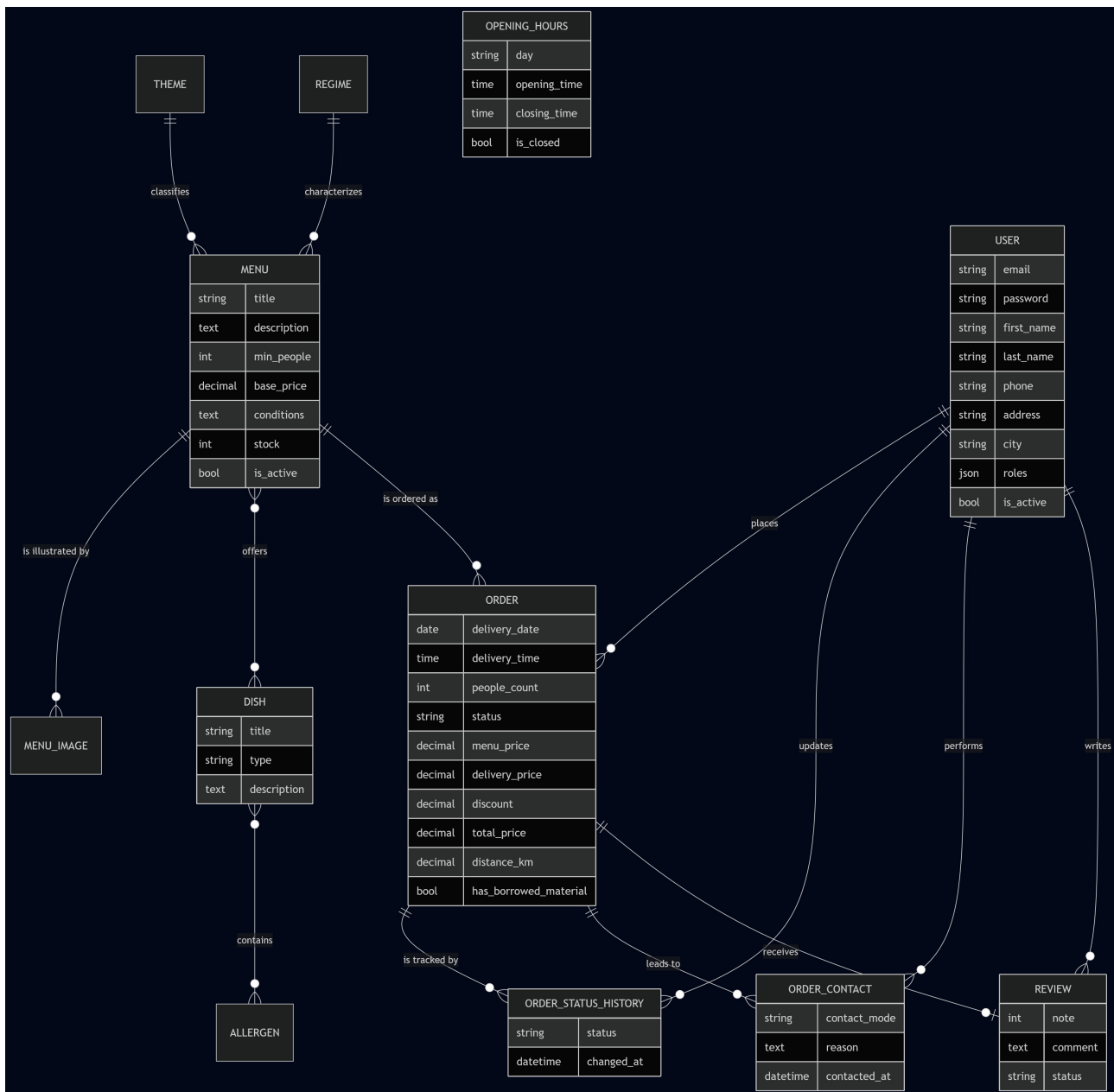
Le modèle de données s'appuie sur le MCD fourni en annexe du cahier des charges, retravaillé pour répondre aux règles métier et aux conventions du framework. Il s'organise en cinq domaines :

- Référentiels : theme (Noël, Pâques, classique, événement), regime (végétarien, vegan, classique), `allergen` et `opening_hours` (horaires d'ouverture affichés en pied de page) ;
- Catalogue : menu (titre, description, nombre de personnes minimum, prix de base, conditions, stock), sa galerie d'images `menu_image`, et les plats dish (entrée, plat ou dessert) ;
- Comptes : user, porteur des trois rôles de l'application (utilisateur, employé, administrateur) ;
- Commandes et suivi : order, son historique de statuts `order_status_history` et la traçabilité des contacts client `order_contact` ;
- Avis : `review`, rattaché à la fois à son auteur et à la commande concernée.

Trois évolutions par rapport au MCD de l'annexe, toutes justifiées par une règle métier ou une convention technique :

- 1.La table `role` a été remplacée par un champ `roles` (JSON) dans `user`, conformément au fonctionnement du composant Security de Symfony. La hiérarchie (`ROLE_ADMIN` hérite de `ROLE_EMPLOYEE`, qui hérite de `ROLE_USER`) est déclarée dans la configuration de sécurité, ce qui évite une jointure à chaque vérification de droits.
- 2.L'avis est rattaché à la commande (et non seulement à l'utilisateur) : le cahier des charges précise qu'un client donne son avis « depuis la commande » une fois celle-ci terminée. Ce rattachement permet de garantir au plus un avis par commande.
- 3.Deux entités de traçabilité ont été ajoutées : `order_status_history` (chaque changement de statut est daté et attribué à son auteur, ce qui alimente le suivi de commande côté client) et `order_contact` (un employé ne peut modifier ou annuler une

commande qu'après avoir contacté le client ; le mode de contact et le motif sont enregistrés).



L'entité `OPENING_HOURS` est volontairement isolée : elle ne partage d'association avec aucune autre, sa vocation étant purement informative (pied de page du site).

## 5.2 Modèle physique et scripts SQL

Le passage au modèle physique produit 14 tables : les 12 tables d'entités et 2 tables d'association (`menu_dish` et `dish_allergen`) matérialisant les relations plusieurs-à-plusieurs. Le schéma complet est livré dans le script `sql/create.sql`, écrit à la main conformément à l'exigence de l'énoncé, et accompagné de `sql/insert.sql` qui charge un jeu de données de démonstration. Les tables sont déclarées dans un ordre garantissant qu'une table est toujours créée avant d'être référencée par une clé étrangère.

Les décisions notables du modèle physique :

- Intégrité déclarative : clés étrangères sur toutes les associations, contrainte `UNIQUE` sur l'email utilisateur, contrainte `UNIQUE` sur la commande référencée par un avis (au plus un avis par commande) et contrainte `CHECK (note BETWEEN 1 AND 5)` qui fait respecter la règle de notation du cahier des charges par la base elle-même, indépendamment de l'application ;
- `ON DELETE CASCADE` limité aux tables d'association : supprimer un menu détache ses plats sans les supprimer, puisqu'un plat peut appartenir à plusieurs menus ; à l'inverse, aucune cascade sur les commandes, dont l'historique doit survivre à tout ;
- Prix figés dans la commande : `menu_price`, `delivery_price`, `discount`, `total_price` et `distance_km` sont enregistrés au moment de la commande. Une modification ultérieure du prix d'un menu n'altère donc jamais les commandes déjà passées, une exigence implicite de toute application marchande ;
- Coordonnées recopiées dans la commande (nom, email, téléphone, adresse) : la commande conserve les informations de livraison telles que saisies au moment T, même si le client modifie ensuite son profil, et le client peut les ajuster commande par commande ;
- Types et encodage : montants en `NUMERIC(10,2)` (jamais de flottants pour l'argent), moteur InnoDB, encodage `utf8mb4` avec collation `utf8mb4_unicode_ci` ; la table `order` est échappée par des backticks, `ORDER` étant un mot réservé de MySQL ;
- Statut de commande : code chaîne alimentant le cycle de vie exigé par le cahier des charges : `PENDING` à la création, puis `ACCEPTED`, `IN_PREPARATION`, `IN_DELIVERY`, `DELIVERED`, `WAITING_MATERIAL` et `COMPLETED`, chaque transition étant historisée dans `order_status_history`.

### 5.3 Modélisation des données statistiques (MongoDB)

Les statistiques de l'espace administrateur proviennent d'une base non relationnelle, comme l'impose le cahier des charges. La collection `order_stats` contient des documents dont la structure est volontairement dénormalisée :

```
{
  "_id": "68f2c41ab7e4d3001c9a1b2f",
  "menuTitle": "Menu de Noël",
  "month": "2026-06-01T00:00:00Z",
  "orderCount": 42,
  "totalRevenue": 1638.00
}
```

**Granularité** : un document par couple (menu, mois). `orderCount` cumule le nombre de couverts commandés et `totalRevenue` le chiffre d'affaires des menus (hors frais de livraison). Cette granularité répond exactement aux besoins de l'écran de statistiques : comparaison des menus

entre eux via un graphique (Chart.js) et calcul du chiffre d'affaires filtré par menu et par période, sans jointure ni agrégation coûteuse à la lecture.

**Alimentation** : à chaque confirmation de commande, l'application met à jour le document du couple (menu, mois) s'il existe, ou le crée sinon. Les données statistiques sont donc toujours à jour sans traitement différé.

**Cohérence entre les deux bases** : c'est le point de vigilance d'une architecture à double persistance. Un écouteur d'événements Doctrine (MenuSyncListener) s'en charge : si un menu est renommé dans MySQL, tous ses documents statistiques MongoDB sont renommés en conséquence ; si un menu est supprimé, ses statistiques sont purgées. La cohérence est ainsi garantie par le code, sans intervention manuelle.

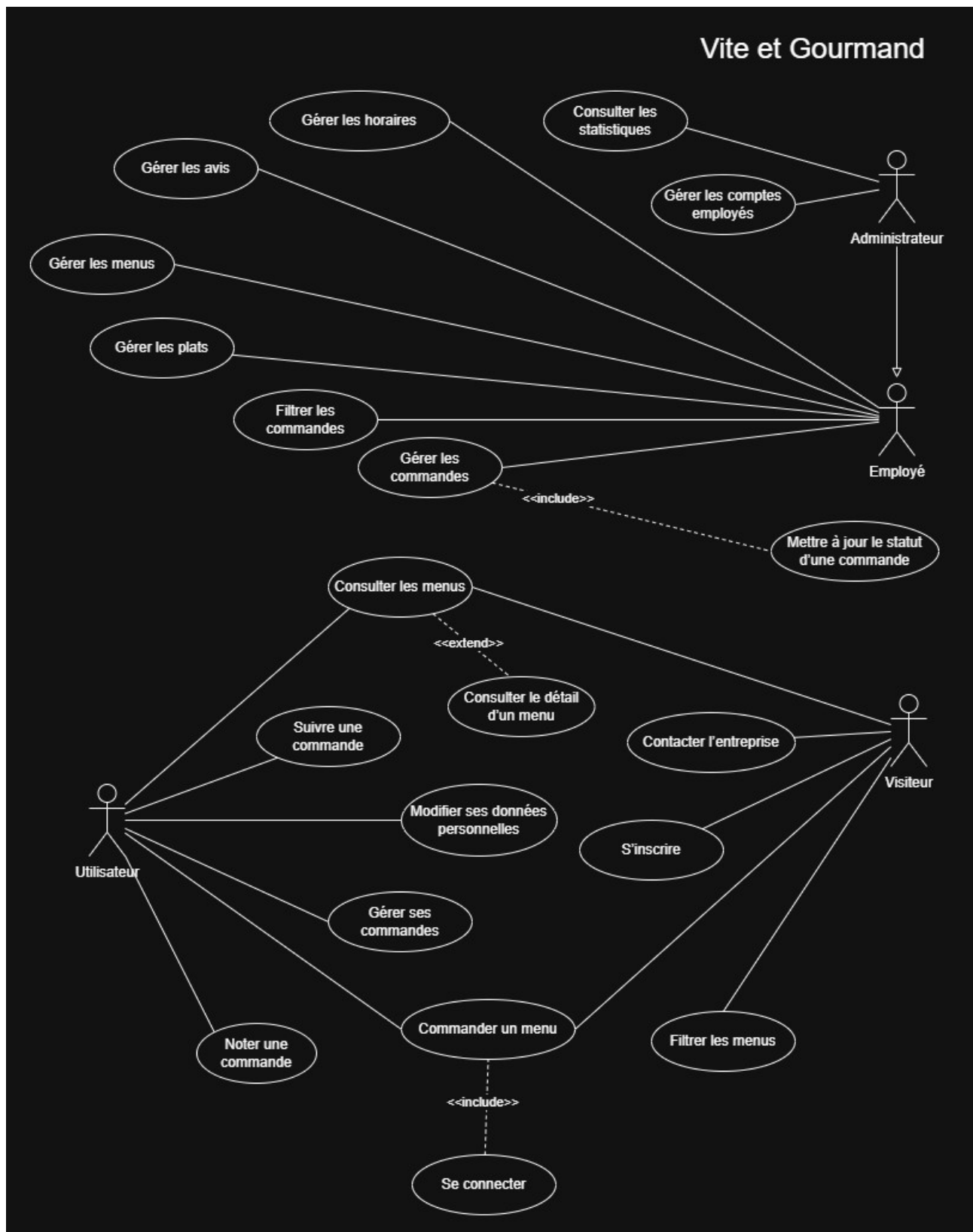
## 5.4 Diagramme de cas d'utilisation

Quatre acteurs interagissent avec le système. Une seule relation de généralisation structure le diagramme : Administrateur hérite d'Employé, ce qui traduit fidèlement l'exigence du cahier des charges selon laquelle l'administrateur doit être capable de faire tout ce qu'un employé peut faire, en plus de ses propres cas d'utilisation (gestion des comptes employés, consultation des statistiques).

Le Visiteur et l'Utilisateur ne sont pas reliés par une généralisation : ce sont deux acteurs distincts, chacun associé à ses propres cas d'utilisation, mais partageant le cas « Commander un menu ». Ce choix de modélisation reflète directement une règle métier du cahier des charges : un visiteur peut initier une commande, mais celle-ci inclut systématiquement « Se connecter » (relation `<<include>>`) ; s'il n'est pas encore inscrit, il devra d'abord créer un compte via « S'inscrire ». Le cas « Consulter le détail d'un menu » étend « Consulter les menus » (relation `<<extend>>`), pour représenter l'accès optionnel au détail depuis la vue globale.

Côté back-office, l'Employé gère les menus, les plats, les horaires et les avis, et traite les commandes ; le cas « Gérer les commandes » inclut « Mettre à jour le statut d'une commande », qui matérialise le cycle de vie détaillé en section 5.2 (de `PENDING` à `COMPLETED`). Un filtre dédié (« Filtrer les commandes ») lui permet de retrouver rapidement les commandes à traiter.





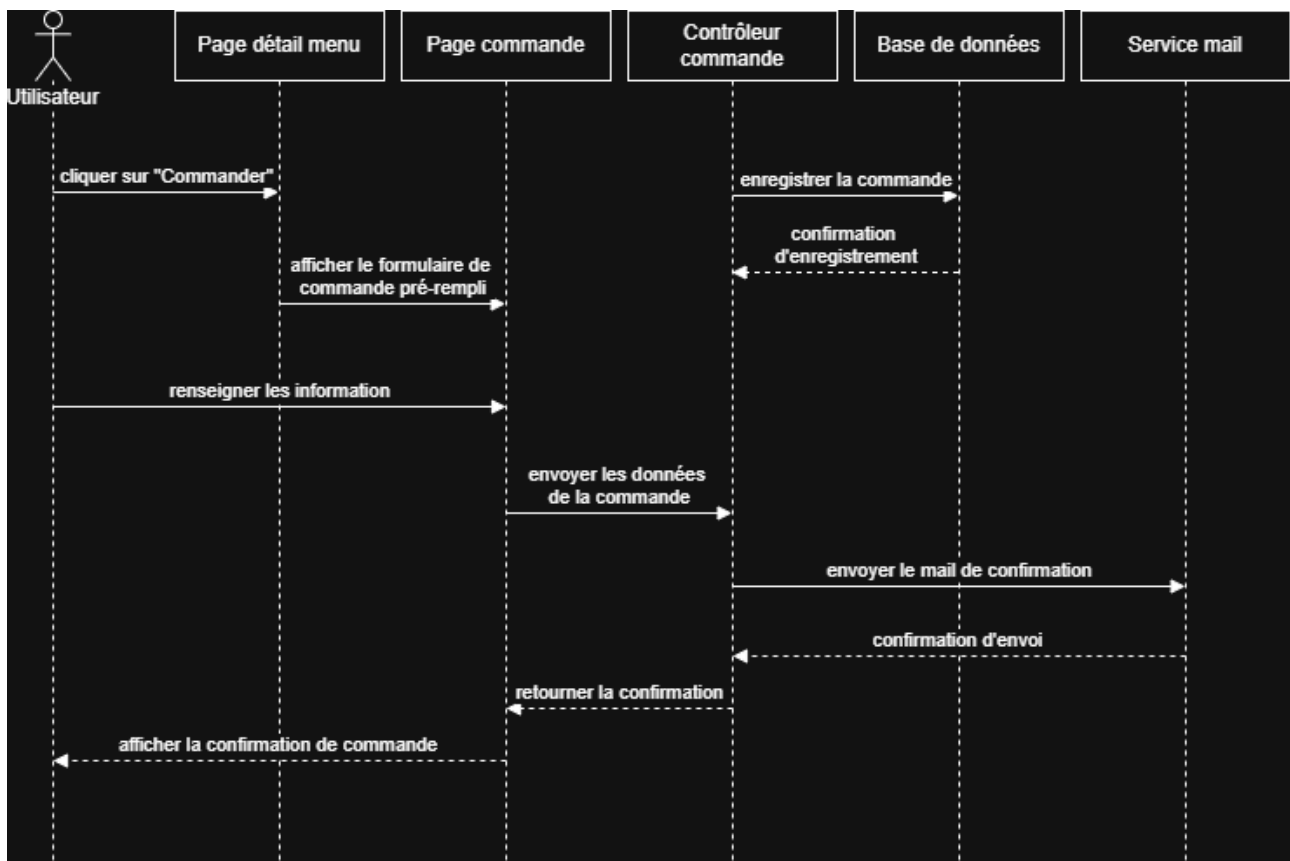
### 5.5 Diagramme de séquence

Le diagramme ci-dessous illustre le parcours de commande d'un menu, depuis le clic sur le bouton « Commander » jusqu'à la confirmation affichée au client. Il met en scène l'utilisateur, les

deux pages concernées (détail du menu et formulaire de commande), le contrôleur de commande, la persistance des données et le service d'envoi de mail.

Le formulaire de commande est pré-rempli à l'ouverture avec les informations du compte de l'utilisateur ; celui-ci complète les champs restants (adresse et date de la prestation notamment), puis le contrôleur enregistre la commande, reçoit confirmation de l'enregistrement, et déclenche l'envoi du mail de confirmation avant de retourner la confirmation au client.

Par souci de lisibilité, le participant « Base de données » représente ici la persistance au sens large. Dans l'implémentation, cette étape se traduit par un enregistrement dans MySQL immédiatement suivi d'une mise à jour des statistiques dans MongoDB (détaillée en sections 5.3 et 6.3), et le calcul du prix par le service de tarification (section 6.3) s'effectue au sein du contrôleur de commande, en amont de l'étape d'enregistrement représentée ici.



Ces détails d'implémentation ont été volontairement simplifiés dans le diagramme, qui se concentre sur l'enchaînement fonctionnel du parcours plutôt que sur le détail technique de la persistance.

## 6. Architecture applicative

### 6.1 Architecture MVC et arborescence du projet

L'application suit une architecture MVC classique côté Symfony, à laquelle s'ajoute une couche service pour isoler la logique métier des contrôleurs :

```
src/
├── Controller/      Point d'entrée HTTP, organisé par zone d'accès (7.1)
│   ├── Public/     Contact, Legal, Main, Menu, Order, Security, User
│   ├── Employee/   Dish, Menu, OpeningHours, Order, Review
│   ├── Admin/      AdminController, StatsController
│   └── Api/        ApiMenuController, ApiOrderController, ApiStatsController
├── Entity/         Modèle relationnel (Doctrine ORM, MySQL)
├── Document/       Modèle documentaire (Doctrine MongoDB ODM)
├── Repository/     Accès aux données, un par Entity ou Document
├── Service/        Logique métier réutilisable (OrderPricingService)
├── EventListener/  Réactions aux événements Doctrine (MenuSyncListener)
├── Security/       UserChecker (7.1)
├── Form/           Types de formulaire Symfony
├── DataFixtures/   Jeu de données de développement (absent de la prod, 9.5)
└── Twig/          Extensions Twig personnalisées (NavigationExtension)

templates/         Vues (Twig), arborescence miroir des contrôleurs
assets/js/         Scripts front, organisés par domaine (menu, order, auth, stats...)
```

L'organisation du dossier `Controller/` reprend exactement les quatre zones d'accès définies dans `security.yaml` (7.1) : `Public/` pour `PUBLIC_ACCESS`, `Employee/` pour `ROLE_EMPLOYEE`, `Admin/` pour `ROLE_ADMIN` et `Api/` pour les routes JSON internes (6.4). Cette correspondance directe entre arborescence de fichiers et politique de sécurité rend immédiat, pour n'importe quel développeur reprenant le projet, de savoir où chercher le code d'une fonctionnalité en fonction du rôle concerné.

Le triptyque MVC se répartit ainsi : le Modèle est porté par `Entity/` et `Document/` (les données) associés à `Repository/` (leur accès) ; la Vue par `templates/`, complétée par `assets/js/` pour les interactions dynamiques (6.4) ; le Contrôleur orchestre la requête sans porter lui-même la logique métier, déléguée aux classes de `Service/`.

## 6.2 Composants d'accès aux données (Doctrine ORM et ODM)

Les deux bases de données coexistent via deux couches Doctrine distinctes mais architecturalement symétriques, chacune configurée par mapping d'attributs PHP plutôt que par YAML ou XML :

```
# config/packages/doctrine.yaml (MySQL, ORM)
mappings:
    App:
        type: attribute
        dir: '%kernel.project_dir%/src/Entity'

# config/packages/doctrine_mongodb.yaml (MongoDB, ODM)
mappings:
    App:
        dir: '%kernel.project_dir%/src/Document'
```

Côté relationnel, une entité comme `Menu` porte ses attributs `#[ORM\Column]` et ses relations `#[ORM\ManyToOne]` / `#[ORM\OneToMany]` directement sur ses propriétés. Côté documentaire, `OrderStat` (la classe qui modélise les documents de la collection `order_stats`, décrite en 5.3) suit exactement le même principe avec les attributs `#[ODM\Document]` et `#[ODM\Field]` :

```
#[ODM\Document(collection: 'order_stats', repositoryClass: OrderStatRepository::class)]
class OrderStat
{
    #[ODM\Id]
    private string $id;

    #[ODM\Field(type: 'string')]
    private string $menuTitle;
    // ...
}
```

Chaque entité et chaque document dispose de son propre dépôt (`Repository`), qui étend `ServiceEntityRepository` pour l'ORM ou `ServiceDocumentRepository` pour l'ODM. C'est dans ces dépôts que vivent les requêtes complexes, jamais dans les contrôleurs. `MenuRepository::findFilteredMenus()` illustre le côté relationnel : un `QueryBuilder` DQL qui enchaîne des jointures (thème, régime, images) et des filtres conditionnels construits dynamiquement selon les paramètres reçus :

```

public function findFilteredMenus(array $filters = []): array
{
    $qb = $this->createQueryBuilder('m')
        ->leftJoin('m.theme', 't')->addSelect('t')
        ->andWhere('m.isActive = :isActive')
        ->setParameter('isActive', true);

    if (isset($filters['minPrice']) && $filters['minPrice'] != "") {
        $qb->andWhere('m.basePrice >= :minPrice')->setParameter('minPrice', $filters['minPrice']);
    }
    // ... même logique pour maxPrice, theme, regime, minPeople, maxPeople

    return $qb->getQuery()->getResult();
}

```

`OrderStatRepository::findByFilters()` illustre le pendant documentaire : la syntaxe change (`field('menuTitle')->in(...)`, `field('month')->gte(...)` propre à l'ODM Mongo), mais le principe reste identique, un dépôt qui encapsule la requête et la restitue à l'appelant sous forme d'objets métier, jamais de tableaux bruts.

### 6.3 Composants métier côté serveur (services, listeners)

`OrderPricingService` (détaillé et testé en 8.2) centralise l'unique règle de tarification de l'application. C'est un service sans état, injecté par autowiring dans tous les contrôleurs qui en ont besoin : le tunnel de commande public (`Public\Order\OrderController::new`) et la modification d'une commande existante (`User\UserController::editOrder`) appellent tous deux `calculatePrices()` de la même façon, garantissant qu'une commande initiale et sa modification obéissent strictement à la même règle de prix.

`MenuSyncListener` matérialise le point de vigilance identifié en 5.3 : garder `order_stats` (MongoDB) cohérent avec `menu` (MySQL) sans intervention manuelle. C'est un écouteur d'événements Doctrine, déclenché automatiquement après une mise à jour ou une suppression d'un `Menu`, via l'attribut `#[AsEntityListener]` :

```

#[AsEntityListener(event: Events::postUpdate, entity: Menu::class)]
#[AsEntityListener(event: Events::postRemove, entity: Menu::class)]
class MenuSyncListener
{
    public function postUpdate(Menu $menu, PostUpdateEventArgs $event): void
    {
        $changeSet = $event->getObjectManager()->getUnitOfWork()->getEntityChangeSet($menu);
        if (!array_key_exists('title', $changeSet)) {
            return;
        }
        [$oldTitle, $newTitle] = $changeSet['title'];

        $this->documentManager->createQueryBuilder(OrderStat::class)
            ->updateMany()
            ->field('menuTitle')->equals($oldTitle)
            ->field('menuTitle')->set($newTitle)

```

```

->getQuery()->execute();
}
}

```

Le listener n'agit que si le titre a effectivement changé (il lit le `changeSet` Doctrine plutôt que de réagir à toute modification du menu), et met à jour en masse tous les documents MongoDB concernés en une seule requête `updateMany`. C'est la preuve concrète, au niveau code, de ce qu'affirme la section 5.3 : la cohérence entre les deux bases est garantie par le code, pas par une procédure manuelle.

L'alimentation initiale de `order_stats` (par opposition à sa synchronisation) n'est pas un listener mais s'effectue directement dans `OrderController::finalize()`, au moment de la confirmation d'une commande : le contrôleur persiste la commande via l'`EntityManager` (MySQL) puis, dans la foulée, recherche ou crée le document du couple (menu, mois) via le `DocumentManager` (MongoDB) et l'incrmente. Les deux gestionnaires Doctrine (ORM et ODM) sont injectés côte à côte dans le même contrôleur, chacun responsable de sa base :

```

$stat = $this->documentManager->getRepository(OrderStat::class)
    ->findOneBy(['menuTitle' => $menu->getTitle(), 'month' => $month]);

if ($stat) {
    $stat->setOrderCount($stat->getOrderCount() + $order->getPeopleCount());
    $stat->setTotalRevenue($stat->getTotalRevenue() + $order->getMenuPrice());
} else {
    $stat = new OrderStat();
    // ...
    $this->documentManager->persist($stat);
}

$this->documentManager->flush();

```

C'est exactement l'étape que le diagramme de séquence (5.5) simplifie sous « enregistrer la commande » / mise à jour MongoDB.

## 6.4 Routes API internes et échanges JSON

Trois contrôleurs exposent des routes JSON internes, consommées en `fetch()` par le JavaScript du site plutôt que par un client tiers : ce sont des routes d'infrastructure interne, pas une API publique documentée.

| Route              | Contrôleur         | Accès                       | Consommateur JS            |
|--------------------|--------------------|-----------------------------|----------------------------|
| GET<br>/api/menus  | ApiMenuController  | PUBLIC_ACCESS               | assets/js/menu/filters.js  |
| GET<br>/api/orders | ApiOrderController | ROLE_EMPLOYEE ([IsGranted]) | assets/js/order/filters.js |
| GET<br>/api/stats  | ApiStatsController | ROLE_ADMIN ([IsGranted])    | assets/js/stats/chart.js   |

Chaque contrôleur reçoit ses filtres en paramètres de requête, délègue au dépôt correspondant (6.2), puis formate manuellement la réponse via une méthode privée (`formatMenu()`, `formatOrder()`) plutôt que de sérialiser l'entité Doctrine brute. Ce choix, délibéré, évite deux

pièges classiques : exposer des champs internes qui n'ont rien à faire côté client, et déclencher des requêtes en cascade non désirées sur des relations chargées à la demande (lazy loading). Le filtrage des menus (exigence de performance en 2.7) illustre le mécanisme de bout en bout. Côté client, chaque changement de curseur ou de select déclenche un nouvel appel, sans rechargement de page :

```
const params = new URLSearchParams();
params.append('minPeople', Math.round(peopleMin));
params.append('maxPrice', Math.round(priceMax));
if (theme) params.append('theme', theme);

const response = await fetch(`/api/menus?${params.toString()}`);
const menus = await response.json();
// menus.forEach(menu => menuList.innerHTML += createMenuCard(menu));
```

Côté serveur, `ApiMenuController::index()` retranscrit ces mêmes paramètres vers `MenuRepository::findFilteredMenus()` (6.2) et renvoie un tableau JSON compact (id, titre, prix, image principale, thème) via `$this->json($data)`, le helper Symfony qui fixe automatiquement l'en-tête `Content-Type: application/json`.

Le tableau de bord statistique (`ApiStatsController`, détaillé en 5.3 et 8.3) suit le même schéma, mais illustre en plus l'agrégation côté client : deux appels `fetch()` sont menés en parallèle (`Promise.all`), un pour le nombre de commandes et un pour le chiffre d'affaires, chacun redessinant son propre graphique Chart.js sans qu'aucune agrégation ne soit recalculée côté serveur, celle-ci étant déjà faite à l'écriture dans `order_stats` (5.3).

## 7. Sécurité

### 7.1 Authentification, rôles et contrôle d'accès

**Rôles et hiérarchie.** Trois rôles structurent l'application : `ROLE_USER`, `ROLE_EMPLOYEE`, `ROLE_ADMIN`. Leur hiérarchie est déclarée une seule fois, dans `config/packages/security.yaml` :

```
role_hierarchy:
    ROLE_EMPLOYEE: [ROLE_USER]
    ROLE_ADMIN: [ROLE_EMPLOYEE, ROLE_USER]
```

Un employé hérite donc de tous les droits d'un utilisateur, et un administrateur de tous les droits d'un employé, exactement la généralisation représentée dans le diagramme de cas d'utilisation (section 5.4). Cette déclaration centralisée évite de dupliquer la logique de rôles dans chaque contrôleur.

**Authentification.** Le firewall `main` utilise le `form_login` natif de Symfony (email et mot de passe transmis sous `login[email]` / `login[password]`), avec un fournisseur d'utilisateurs Doctrine adossé à l'entité `User` (identifiant : l'email). Les mots de passe sont hashés via `password_hashers: ... 'auto'`, qui laisse Symfony choisir l'algorithme le plus robuste

disponible (bcrypt ou argon2 selon l'environnement) : aucun mot de passe n'est jamais stocké ou comparé en clair.

Un `UserChecker` (`src/Security/UserChecker.php`) s'exécute avant l'authentification et bloque toute tentative de connexion sur un compte dont le champ `isActive` est à `false`, avec un message explicite. C'est ce mécanisme qui rend utile la fonctionnalité « désactiver un compte employé » de l'espace administrateur (`AdminController::employeeToggle`) : désactiver un compte ne le supprime pas, mais lui interdit immédiatement toute connexion.

Contrôle d'accès à deux niveaux. Le premier niveau, déclaratif, filtre par préfixe d'URL dans `security.yaml`

| Préfixe                                                                                     | Rôle requis                |
|---------------------------------------------------------------------------------------------|----------------------------|
| <code>/mon-espace</code>                                                                    | <code>ROLE_USER</code>     |
| <code>/espace-employe</code>                                                                | <code>ROLE_EMPLOYEE</code> |
| <code>/espace-admin</code>                                                                  | <code>ROLE_ADMIN</code>    |
| <code>/connexion, /inscription, /mot-de-passe-oublie, /reinitialisation-mot-de-passe</code> | <code>PUBLIC_ACCESS</code> |
| <code>/</code>                                                                              | <code>PUBLIC_ACCESS</code> |

Le second niveau, plus fin, protège individuellement certains points d'entrée API par l'attribut `#[IsGranted(...)]`, indépendamment de leur URL : `ApiStatsController` exige `ROLE_ADMIN`, `ApiOrderController` exige `ROLE_EMPLOYEE`. Cette double protection (filtrage large par le firewall, vérification ciblée sur les endpoints sensibles) limite le risque d'un contrôle d'accès oublié sur une route ajoutée après coup.

Les actions destructrices ou irréversibles hors formulaire Symfony (annulation de commande côté client, suppression ou désactivation d'un compte employé) sont en plus protégées individuellement par un jeton CSRF vérifié manuellement (`isCsrfTokenValid()`), détaillé en 7.2.

**Réinitialisation de mot de passe.** Le flux (`ResetPasswordController`) génère un jeton `UUID v4` stocké sur l'utilisateur avec une expiration d'une heure (`resetTokenExpiresAt`). Le message affiché après une demande est volontairement identique, qu'un compte existe ou non pour l'email saisi (« Si un compte existe avec cet email, un lien... ») : ce choix empêche un attaquant de déduire, à partir des réponses du formulaire, quels emails sont inscrits sur le site (protection contre l'énumération de comptes).

### 7.3 Données personnelles et conformités RGPD

Données collectées et finalité. L'application collecte, à l'inscription et lors d'une commande : nom, prénom, email, téléphone, adresse et ville. La page `/mentions-legales` (`LegalController`)

précise explicitement la finalité (« gestion des commandes et de la relation client ») et l'absence de transmission à des tiers, deux obligations d'information du RGPD.

Une particularité du modèle de données (déjà justifiée en 5.2 pour des raisons de traçabilité commerciale) a aussi une portée RGPD : les coordonnées sont recopiées dans chaque commande plutôt que simplement référencées depuis le profil. Cela signifie que même après une modification du profil utilisateur, l'historique des commandes conserve les données telles que saisies au moment de l'achat, un point à mentionner si un client exerce un droit de rectification (la correction s'applique à son profil, pas rétroactivement à ses commandes passées).

Exercice des droits. Le RGPD garantit un droit d'accès, de rectification et de suppression. Dans l'état actuel de l'application, ces droits ne sont pas self-service : la page mentions légales invite le client à contacter `contact@vite-et-gourmand.fr` et un traitement manuel en base est nécessaire pour toute demande de suppression de compte utilisateur (aucune route de suppression de compte n'existe côté `UserController`, seule la suppression d'un compte employé existe côté back-office).

C'est un choix de dimensionnement cohérent avec une TPE de deux gérants plutôt qu'une lacune cachée, mais c'est aussi une limite réelle à assumer dans le manuel utilisateur et, le cas échéant, devant le jury : une demande de droit à l'oubli demande aujourd'hui une intervention technique directe en base, en tenant compte de l'absence de cascade sur les commandes (section 5.2), pour ne pas casser l'historique des commandes déjà passées.

Cookies. Vérification faite sur l'ensemble des templates : aucun script de mesure d'audience ou de traçage tiers (Google Analytics, Google Tag Manager, Matomo, pixel Facebook) n'est présent. Le seul cookie émis est le cookie de session PHP, strictement nécessaire au maintien de la connexion. Conformément aux recommandations de la CNIL, les cookies strictement nécessaires au fonctionnement du service sont exemptés de consentement préalable : l'absence de bandeau cookies sur le site est donc conforme, et non un oubli.

Sécurité des données au repos. Les mots de passe sont systématiquement hashés (7.1) : en cas de compromission de la base, aucun mot de passe en clair ne serait exposé. Ce n'est pas le cas de la variable d'environnement `DATABASE_URL` de l'environnement de production, voir l'alerte en tête de ce message et le détail en 9.4.

## **7.4 Veille sécurité**

La sécurité de l'application ne s'arrête pas à sa mise en production ; une veille est organisée autour de trois pratiques :

- le suivi des Symfony Security Advisories, le canal officiel par lequel l'équipe Symfony publie les vulnérabilités découvertes dans le framework et leurs correctifs ;
- l'exécution régulière de `composer audit`, qui confronte les versions exactes des dépendances du projet (fichier `composer.lock`) à la base de données publique des vulnérabilités PHP et signale toute bibliothèque compromise ;

- la mise à jour régulière des dépendances dans les bornes de version définies, facilitée par le calendrier de maintenance prévisible de Symfony (versions LTS).

## 8. Tests et jeu d'essai

### 8.1 Stratégie de test

Le projet ne dispose pas de suite de tests automatisés (aucun répertoire `tests/`, aucune configuration PHPUnit propre au projet à ce stade) : c'est un choix méthodologique assumé, dicté par le calendrier de l'ECF, et non un oubli à dissimuler. La stratégie retenue repose sur des campagnes de tests manuels fonctionnels, menées directement dans le navigateur, parcours par parcours et rôle par rôle (visiteur, utilisateur, employé, administrateur), en utilisant le diagramme de cas d'utilisation (section 5.4) comme grille de couverture : chaque cas d'utilisation a fait l'objet d'un passage manuel avant validation.

Le modèle physique (section 5.2) a lui fait l'objet d'une validation outillée : le script `sql/create.sql` a été exécuté sur une base MySQL jetable (conteneur dédié, base vierge), puis confronté au mapping Doctrine via `doctrine:schema:validate`, ce qui a permis de détecter et documenter les seuls écarts volontaires (renommages cosmétiques d'index, contrainte `uq_user_email`) plutôt que des erreurs.

La règle métier la plus complexe de l'application, le calcul du prix d'une commande (`OrderPricingService`), fait l'objet d'un jeu d'essai détaillé et chiffré ci-dessous plutôt que d'un test manuel simple, car elle combine plusieurs conditions (remise, franchise de livraison, distance).

### 8.2 Jeu d'essai : calcul du prix d'une commande

Règle testée, telle qu'implémentée dans `OrderPricingService::calculatePrices()` et confirmée par les CGV (`templates/public/legal/cgv.html.twig`) :

- `prix_menu = prix_base_menu × nombre_de_personnes` ;
- `frais_livraison = 0` si la ville est Bordeaux, sinon `5 + 0,59 × distance_km` (distance calculée par géocodage de l'adresse via l'API Nominatim, puis formule de Haversine depuis un point de référence à Bordeaux) ;
- `remise = 10 % du prix_menu` si `nombre_de_personnes ≥ minimum_du_menu + 5`, sinon `0` ;
- `prix_total = prix_menu - remise + frais_livraison`.

Méthode. Pour garantir l'exactitude des montants ci-dessous, l'algorithme a été extrait et rejoué tel quel (mêmes formules, mêmes constantes) avec des données réelles issues de `sql/insert.sql` (menu « Dolce Vita », id 4 : prix de base 24,00 €, minimum 4 personnes). Le point de départ du calcul de distance (44.837789, -0.57918) est celui codé en dur dans le service.

Le scénario B utilise des coordonnées illustratives pour Mérignac, l'appel réseau réel à Nominatim n'étant pas disponible depuis l'environnement où ce document est rédigé ; en usage normal, ces coordonnées proviennent d'un géocodage en direct au moment de la commande.

**Méthode.** Pour garantir l'exactitude des montants ci-dessous, l'algorithme a été extrait et rejoué tel quel (mêmes formules, mêmes constantes) avec des données réelles issues de `sql/insert.sql` (menu « Dolce Vita », id 4 : prix de base 24,00 €, minimum 4 personnes). Le point de départ du calcul de distance (44.837789, -0.57918) est celui codé en dur dans le service.

Le scénario B utilise des coordonnées illustratives pour Mérignac, l'appel réseau réel à Nominatim n'étant pas disponible depuis l'environnement où ce document est rédigé ; en usage normal, ces coordonnées proviennent d'un géocodage en direct au moment de la commande.

|                 | Scénario A : Bordeaux                            | Scénario B : Mérignac                                  |
|-----------------|--------------------------------------------------|--------------------------------------------------------|
| Menu            | Dolce Vita (24,00 €, min. 4 pers.)               | Dolce Vita (24,00 €, min. 4 pers.)                     |
| Convives        | 9 ( $\geq 4 + 5$ , remise applicable)            | 6 ( $< 9$ , pas de remise)                             |
| Ville           | Bordeaux                                         | Mérignac                                               |
| Distance        | 0 km (livraison offerte)                         | $\approx 5,61$ km                                      |
| Prix menu       | $24,00 \times 9 = \mathbf{216,00 \text{ €}}$     | $24,00 \times 6 = \mathbf{144,00 \text{ €}}$           |
| Remise          | $216,00 \times 10 \% = \mathbf{21,60 \text{ €}}$ | <b>0,00 €</b>                                          |
| Frais livraison | <b>0,00 €</b>                                    | $5 + 0,59 \times 5,61 \approx \mathbf{8,31 \text{ €}}$ |
| Prix total      | $216,00 - 21,60 + 0 = \mathbf{194,40 \text{ €}}$ | $144,00 - 0 + 8,31 = \mathbf{152,31 \text{ €}}$        |

**Résultat attendu vs obtenu** : les deux scénarios produisent exactement les montants attendus au regard des CGV (remise à 10 % correctement déclenchée au seuil minimum + 5, franchise de livraison correctement appliquée sur Bordeaux uniquement). Aucun écart constaté sur ce test.

### 8.3 Analyse des écarts

Quatre écarts ou points d'attention ont été identifiés au cours des campagnes de test manuelles, aucun ne remettant en cause le fonctionnement métier :

**1.** Absence de tests automatisés (8.1) : limite méthodologique assumée. Une non-régression n'est aujourd'hui détectée que par un nouveau passage manuel, ce qui ralentit la validation d'une modification future du calcul de prix ou du cycle de statut de commande.

**2.** `/api/stats` renvoie des tableaux vides sans le paramètre `menus[]` : à première vue, un testeur non averti peut croire à un bug (endpoint « qui ne retourne rien »). C'est en réalité un choix volontaire de `ApiStatsController` (le graphique attend une sélection explicite de menus avant de tracer quoi que ce soit). Test de validation : `GET /api/stats?menus[]=Dolce Vita` (en tant qu'administrateur) retourne bien un jeu de données structuré pour Chart.js. Point à clarifier dans le manuel utilisateur pour éviter une fausse alerte en soutenance.

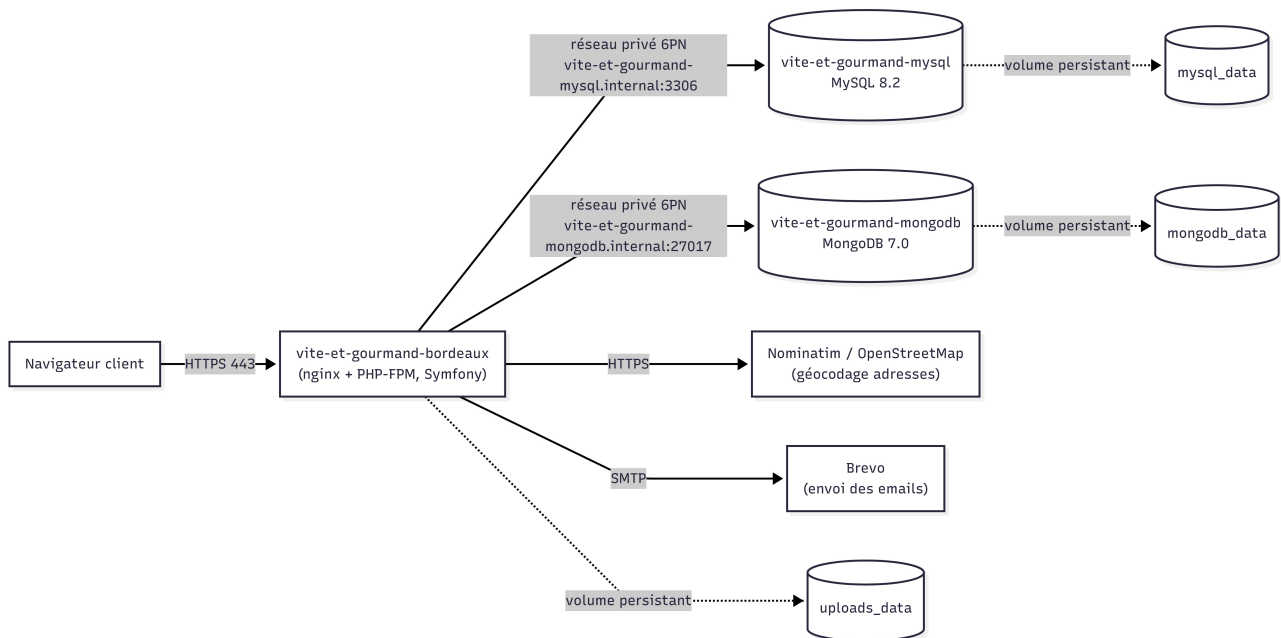
**3.**Incohérences connues dans le jeu de données de démonstration (`sql/insert.sql`, miroir des `DataFixtures`) : un avis daté d'avril pour une commande de mai, un historique de statut antérieur à la date de livraison de la commande 1, une image de menu référencée en base mais absente de `public/images/menus/` pour le menu 2 (« Tradition Française », lui-même inactif : `is_active = 0`, donc non exposé publiquement). Ces incohérences sont héritées du jeu de données de développement, assumées et sans impact sur le fonctionnement réel de l'application, mais méritent d'être mentionnées en annexe pour ne pas être découvertes en direct par le jury.

**4.**Dépendance à un service de géocodage externe gratuit (Nominatim/OpenStreetMap) pour le calcul de la distance de livraison : en cas d'indisponibilité du service ou d'adresse non reconnue, `calculatePrices()` retourne `null` et le formulaire affiche une erreur métier propre (« Adresse introuvable »), ce qui est le comportement attendu, mais introduit une dépendance externe hors du contrôle de l'application, à surveiller en production.

## 9. Documentation du déploiement

### 9.1 Architecture de production : trois applications Fly.io

L'application est déployée sur Fly.io sous la forme de **trois applications distinctes**, plutôt qu'un unique conteneur monolithique, toutes dans la région `cdg` (Paris, proche des utilisateurs français) :



Les trois applications communiquent exclusivement via le réseau privé interne de Fly (`6PN`, IPv6 uniquement, hostnames en `.internal`) : les bases de données ne sont jamais exposées directement sur Internet, seule l'application PHP l'est (`http_service` avec `force_https = true`). Ce choix de séparation, plutôt qu'un unique conteneur tout-en-un, permet de faire évoluer, redémarrer ou sauvegarder chaque brique indépendamment, au prix d'une configuration réseau un peu plus complexe (détaillée en 9.2).

### 9.2 Préparation : Dockerfiles et configuration Fly

Application PHP (`fly/php/Dockerfile`) : construction en trois étapes (multi-stage build), pour garder l'image finale la plus légère possible :

1. `node:20-alpine` compile les assets front (Webpack Encore) ;
2. `composer:2` installe les dépendances PHP en mode production (`--no-dev --optimize-autoloader`, extension `mongodb` ignorée à l'installation puis ajoutée manuellement dans l'image finale) ;
3. l'image finale `php:8.3-fpm-alpine` embarque nginx et PHP-FPM, orchestrés ensemble par supervisord dans un seul conteneur (nginx sert les fichiers statiques et transmet le PHP à PHP-FPM en local sur `127.0.0.1:9000`), avec un cache OPcache réchauffé au moment du build (`cache:warmup`).

Bases de données : deux Dockerfiles minimalistes, l'un basé sur `mysql:8.2` (avec un forçage de l'encodage `utf8mb4` via `charset.cnf`, cohérent avec le choix du modèle physique en 5.2), l'autre sur `mongo:7.0` (démarré avec `--ipv6 --bind_ip_all`, condition nécessaire pour être joignable depuis le réseau privé IPv6 de Fly).

Fichiers `fly.toml` : chaque application déclare son propre `fly.toml` (un par dossier `fly/php`, `fly/mysql`, `fly/mongodb`), avec pour l'app PHP un `[[mounts]]` vers `/app/public/uploads` (persistance des images uploadées entre redéploiements) et un `[http_service]` avec health check GET `/` toutes les 15 secondes ; pour les deux bases, un `[[mounts]]` vers leur répertoire de données respectif `/var/lib/mysql`, `/data/db`.

### 9.3 Étapes du déploiement (application, MySQL, MongoDB)

1. Création des trois applications Fly et de leurs volumes persistants associés (`mysql_data`, `mongodb_data`, `uploads_data`) dans la région `cdg`
2. Déploiement des deux bases de données en premier (`fly deploy` depuis `fly/mysql` puis `fly/mongodb`), pour qu'elles soient joignables au démarrage de l'application.
3. Création du schéma MySQL en exécutant directement `sql/create.sql` sur l'instance de production (le script `handwritten` est la source de vérité du schéma, cohérent avec l'exigence de l'énoncé). Les migrations Doctrine présentes dans `migrations/` sont des artefacts générés en cours de développement local ; elles ne sont pas rejouées automatiquement en production (le `release_command` correspondant est volontairement laissé en commentaire dans `fly/php/fly.toml`).
4. Déploiement de l'application PHP (`fly deploy` depuis `fly/php`), qui construit l'image multi-stage et bascule le trafic une fois le health check `/` positif.
5. Vérification du health check intégré et des premiers accès publics.

### 9.4 Gestion des secrets et variables d'environnement

Fly.io propose deux mécanismes pour transmettre des variables à une application : le bloc `[env]` du `fly.toml` (en clair, versionné avec le code) pour la configuration non sensible, et `fly secrets set` (chiffré, jamais versionné) pour les identifiants et clés.

D'après le suivi de projet, les secrets applicatifs sensibles (`MAILER_DSN`, `MAILER_FROM`, `CONTACT_EMAIL`) ont bien été posés via `fly secrets set`, en cohérence avec la centralisation de l'expéditeur mail dans `config/packages/mailer.yaml` (`%env(MAILER_DSN)%`, aucun `->from()` codé en dur dans les contrôleurs). L'envoi réel des emails en production passe par le SMTP de Brevo.

Point de vigilance identifié pendant la rédaction de ce document : `DATABASE_URL`, incluant l'identifiant et le mot de passe MySQL en clair, était déclaré dans le bloc `[env]` de

`fly/php/fly.toml`, donc versionné et visible sur le dépôt GitHub public, au lieu d'être posé via `fly secrets set` comme les autres identifiants sensibles.

Ce point a été corrigé selon la démarche suivante : rotation du mot de passe de l'utilisateur MySQL en production (`ALTER USER`), migration de `DATABASE_URL` vers un secret Fly (`fly secrets set DATABASE_URL=...`), et remplacement de la valeur en clair dans `fly.toml` par un placeholder `!ChangeMe!`, cohérent avec la convention déjà utilisée dans le `.env` versionné.

Un point d'attention technique a été rencontré à cette occasion : la configuration Doctrine utilisant `%env(resolve:DATABASE_URL)%` (voir 6.2), tout caractère `%` présent dans le mot de passe (issu de l'encodage URL des caractères spéciaux du mot de passe) doit être échappé en `%%` dans le secret, sans quoi Symfony tente de le résoudre comme une référence à un paramètre du conteneur.

Le secret `MYSQL_PASSWORD` de l'application MySQL elle-même a également été resynchronisé, par cohérence, bien que le volume de données persistant (9.1) empêche déjà toute réinitialisation involontaire du mot de passe au redémarrage.

Cet incident illustre l'importance de ne jamais committer de secret en clair, même dans un fichier de configuration d'infrastructure : la correction a nécessité non seulement un changement de configuration, mais également une rotation réelle du mot de passe exposé, celui-ci devant être considéré comme compromis dès lors qu'il apparaît dans l'historique d'un dépôt public.

## 9.5 Initialisation des données en production

Le jeu de données réel de production n'est pas celui de développement : `sql/insert.sql` est un jeu de démonstration destiné aux environnements locaux (12 utilisateurs, 24 plats, 10 menus, 8 commandes, 6 avis, miroir des `DataFixtures`), volontairement absent de l'image de production puisque le build s'effectue en `--no-dev` (les fixtures Doctrine, qui en dépendent, ne sont pas disponibles dans cette image).

En production, les données de catalogue (menus, plats, thèmes, régimes) sont donc saisies directement via le back-office employé/administrateur de l'application. Les statistiques MongoDB (`order_stats`) ont quant à elles été initialisées manuellement via `mongosh` en recopiant le contenu de la base locale vers l'instance de production après purge des données de démonstration. Ce mode d'initialisation manuel introduit un point d'attention documenté : la correspondance entre un document de statistiques et un menu se fait sur le titre exact du menu, donc tout écart de libellé entre la saisie back-office et l'historique statistique romprait le rapprochement.

## 9.6 Vérifications post-déploiement

- Santé applicative : health check HTTP intégré à Fly (`GET` / toutes les 15 secondes), avec redémarrage automatique de la machine en cas d'échec.

- Parcours fonctionnel : connexion sur chacun des trois rôles, passage d'une commande complète (avec calcul de prix réel, geocoding en direct), vérification de la réception effective des emails transactionnels (bienvenue, confirmation, réinitialisation de mot de passe) via Brevo.
- Persistance des données : vérification que les images uploadées survivent à un redéploiement (volume `uploads_data`), et que les données MySQL/MongoDB survivent également (volumes `mysql_data`, `mongodb_data`).
- Tableau de bord statistiques : test explicite avec le paramètre `menus[]` renseigné (voir écart 8.3, point 2), pour confirmer que les données saisies manuellement en base Mongo remontent correctement dans Chart.js.

## 10. Annexes

### 10.1 Glossaire

| Terme                     | Définition                                                                                                                                                                                  |
|---------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>MCD</b>                | Modèle Conceptuel de Données : représentation des données indépendante de toute technologie, exprimée dans le langage métier du projet.                                                     |
| <b>MPD</b>                | Modèle Physique de Données : traduction technique du MCD en tables, colonnes, types et contraintes réellement implémentés en base.                                                          |
| <b>ORM</b>                | Object-Relational Mapping : correspondance entre objets du code (entités PHP) et lignes d'une base relationnelle ; Doctrine est l'ORM utilisé par Symfony.                                  |
| <b>DQL</b>                | Doctrine Query Language : langage de requêtage orienté objet de Doctrine, traduit en SQL, qui évite d'écrire du SQL brut dans le code.                                                      |
| <b>CSRF</b>               | Cross-Site Request Forgery : attaque exploitant la session active d'un utilisateur pour déclencher une action à son insu depuis un autre site ; contrée par un jeton unique par formulaire. |
| <b>XSS</b>                | Cross-Site Scripting : injection de code exécutable (JavaScript) dans une page via une saisie utilisateur non échappée.                                                                     |
| <b>RGPD</b>               | Règlement Général sur la Protection des Données : cadre européen encadrant la collecte et le traitement des données personnelles.                                                           |
| <b>Firewall (Symfony)</b> | Zone du site sur laquelle s'applique une configuration d'authentification et de contrôle d'accès donnée.                                                                                    |

| Terme                       | Définition                                                                                                                               |
|-----------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| <b>6PN</b>                  | Réseau privé IPv6 interne de Fly.io, reliant les applications d'un même compte sans exposition publique.                                 |
| <b>Geocoding</b>            | Conversion d'une adresse postale en coordonnées géographiques (latitude/longitude), utilisée ici pour calculer les frais de livraison.   |
| <b>Formule de Haversine</b> | Formule trigonométrique donnant la distance à vol d'oiseau entre deux points connaissant leurs coordonnées GPS.                          |
| <b>Webpack Encore</b>       | Surcouche Symfony de Webpack, utilisée pour compiler et emballer les assets front (JS, CSS).                                             |
| <b>Fixtures</b>             | Jeu de données de test généré par code (Doctrine <code>DataFixtures</code> ), utilisé en développement, absent des images de production. |
| <b>TPE</b>                  | Très Petite Entreprise (ici, l'entreprise cliente Vite & Gourmand, deux co-gérants).                                                     |

## 10.2 Références

- Dépôt GitHub : [github.com/Hatenaa/Vite-Gourmand](https://github.com/Hatenaa/Vite-Gourmand)
- Application en production : [vite-et-gourmand-bordeaux.fly.dev](https://vite-et-gourmand-bordeaux.fly.dev)
- Services externes intégrés : [Nominatim / OpenStreetMap](#) (géocodage des adresses de livraison), Brevo (envoi des emails transactionnels en production)